BSDA2001P

INTRODUCTION TO DEEP LEARNING & GENERATIVE AI

Project Report

Messy Mashup: Music Genre Classification

Student ID: 23f3003805

Term: T1 2026

Course: DL & GenAI Project

Project Links

WandB Dashboard: https://wandb.ai/23f3003805-dl-genai-project/23f3003805-t12026

HuggingFace: https://huggingface.co/spaces/shreya585/dl-genai-audio-messy-mashup

GitHub Repository: https://github.com/23f3003805/dl-genai-project-25-t3

1. ABSTRACT SUMMARY

This report documents the complete development lifecycle of a deep learning system built to solve the Messy Mashup Kaggle competition: a challenging music genre classification task under realistic and noisy mixing conditions. The competition required participants to classify audio mashups — recordings formed by combining instrument stems from different songs of the same genre — into one of ten distinct genre categories, further complicated by additive environmental noise from the ESC-50 dataset.

The core challenge was a distribution shift between training and test data. While training data consisted of clean, separated instrument stems (drums, vocals, bass, others), the test set comprised synthesized mashups with cross-song mixing, tempo variations, and injected noise. Bridging this gap required a deliberate data augmentation strategy that simulated test-like conditions during training.

Four models were developed and evaluated: a baseline SimpleCNN, a hybrid Convolutional Recurrent Neural Network (CRNN), a pre-trained EfficientNet-B0 adapted for audio classification, and a final augmented AudioClassifier (EfficientNet-B0 with aggressive SpecAugment). All models operated on Mel Spectrogram representations of audio. The final ensemble prediction pipeline combined outputs from all four models using majority voting to generate the competition submission. Training experiments were tracked in real-time via Weights and Biases (WandB). The trained models were deployed as an interactive inference application on Hugging Face Spaces. The primary evaluation metric used throughout was the Macro F1 Score, consistent with the competition's evaluation criteria.

2. INTRODUCTION

2.1. Problem Statement

The Messy Mashup competition, hosted on Kaggle platform, presents a music genre classification problem with a deliberate adversarial twist of classifying audio that has been artificially reconstructed from multiple sources. Specifically:

- Training data: Clean, instrument-separated stems (drums.wav, vocals.wav, bass.wav, others.wav) organized by genre across 10 categories.

- Test data: Mashup audio formed by combining stems from different songs of the same genre, with possible tempo synchronization changes and additive noise from the ESC-50 environmental sound dataset injected at varying intensities.
- 10 target genres: blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock.

2.2. Project Objective

The primary goals of this project were:

- Build models that generalize from clean stem data to noisy mashup test data by simulating test-like inputs during training.
- Explore and compare multiple deep learning architectures ranging from simple CNNs to pre-trained transformer-style backbones.
- Gain hands-on experience with audio processing libraries (librosa, torchaudio) and the HuggingFace ecosystem.
- Track all experiments systematically using Weights and Biases for transparent, reproducible research.
- Deploy the trained model as a working inference application on Hugging Face Spaces.

2.3. Report Structure

The report is structured as follows: Section 3 covers dataset description and preprocessing strategies; Section 4 explains the tokenization (feature extraction) pipeline; Section 5 details all four model architectures; Section 6 presents performance comparisons and analysis; Section 7 discusses deployment; and Section 8 concludes with learnings and future directions

3. DATASET AND PREPROCESSING

3.1. Dataset Description

The competition dataset consisted of following components -

genres_stems/	Songs organized by genre; each song folder contains bass.wav, drums.wav, vocals.wav, others.wav

mashups/	Pre-built mashup audio files for the test set
ESC-50-master/audio/	2000 environmental sounds for noise injection augmentation
test.csv	List of test file paths and IDs for submission
sample_submission.csv	Submission format template

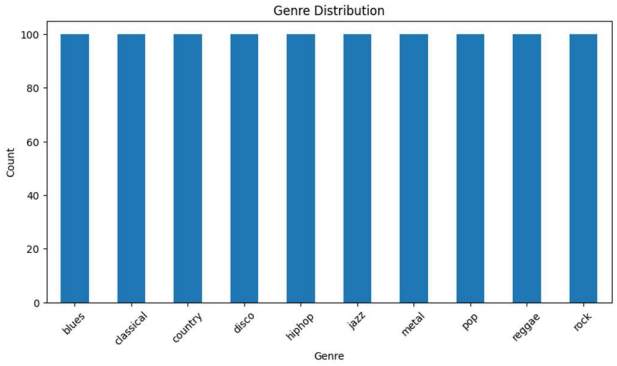
The training corpus covered 10 genres: blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock. Each genre folder contained multiple songs, each decomposed into four stems. This amounted to several hundred songs per genre, providing a rich but indirect representation of each genre's sonic identity.

3.2. Exploratory Data Analysis

The following were performed Exploratory Data Analysis before training-

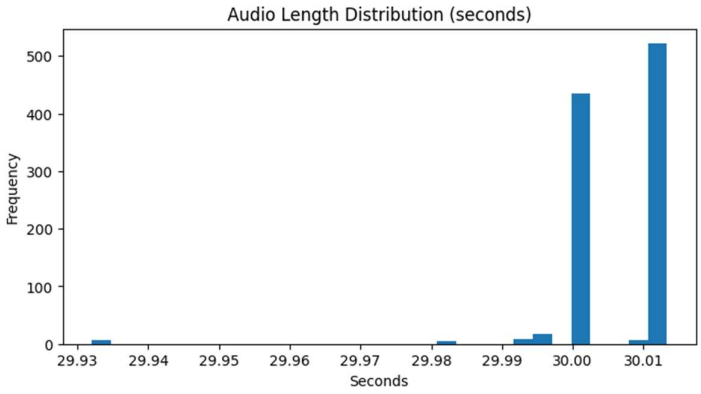
3.2.1. Genre Distribution

A bar chart of genre distribution confirmed that the dataset was approximately balanced across all 10 classes. No significant class imbalance was observed, which means standard cross-entropy loss was appropriate without needing class weighting.



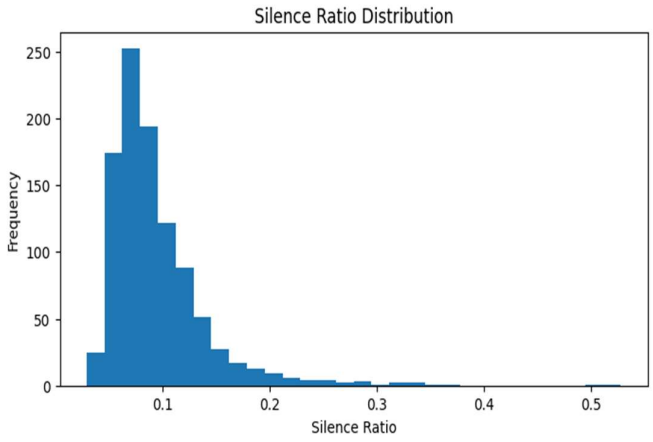
3.2.2. Audio Length Analysis

Audio lengths varied across songs. After constructing synthetic mashups, the distribution of lengths was plotted. Most mashups ranged from 30 to 120 seconds. A fixed crop size of 6 seconds (SR * 6 = 22050 * 6 = 132,300 samples) was chosen to standardize input length while preserving enough temporal context for genre recognition.



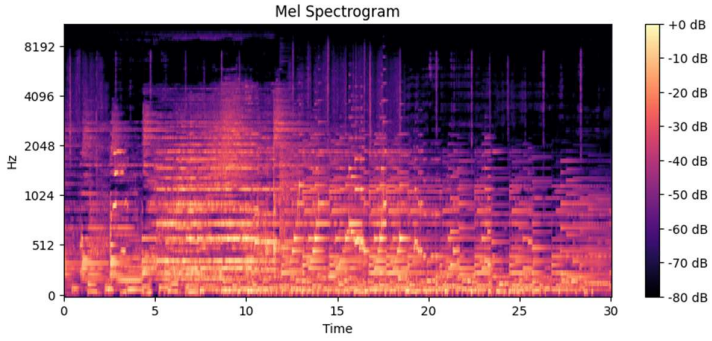
3.2.3. Silence Analysis

A silence ratio was computed for each synthesized mashup by identifying samples below an amplitude threshold of 0.01. The average silence ratio across the dataset was around 20-30%, which is expected for instrument stems where certain instruments may be inactive for extended stretches. This informed the decision to use random cropping rather than fixed-start cropping, ensuring that silent regions would not dominate a training sample.



3.2.4. Spectrogram Visualisation

Mel spectrograms were visualized for sample mashups from different genres. Classical tracks showed concentrated energy in mid-frequency bands with smooth transitions. Metal tracks showed broader energy distribution across high frequencies. These patterns confirmed that Mel Spectrograms were an informative representation for genre discrimination.



3.2.5. File Size Analysis

All stems were checked for potentially corrupt or near-empty files. Files below 4KB were flagged as potentially silent or trivially short. This identified a small number of problematic files that were naturally excluded from training by the silence filtering step.

3.3. Synthetic Mashup Generation

Since the test data consisted of mashups while training data consisted of clean stems, a synthetic mashup generator was implemented to close this distribution gap. synthetic_mashup() function:

- Sampled 4 different songs of the same genre.
- Loaded one stem type (bass, drums, vocals, others) from each different song.
- Mixed the four stems together by simple waveform addition, normalizing to prevent clipping.
- This simulated the cross-song stem recombination that characterizes the test set.

By training on synthetic mashups rather than individual clean stems, the model was exposed to inputs that closely resembled the test distribution, even though no test-time data was used.

3.4. Data Augmentation

Four augmentation techniques were applied during training to improve generalization:

3.4.1. Noise Injection

A random ESC-50 noise file was selected and mixed with the audio signal at a randomly chosen Signal-to-Noise Ratio (SNR). The noise was padded if shorter than the signal, and properly scaled to achieve the target SNR. This replicated the noise injection applied in the test set generation process.

3.4.2. Time Stretching

Applied with 50% probability, stretching the audio by a factor sampled uniformly from [0.9, 1.1]. This simulated the tempo variations introduced during stem synchronization in the test mashups.

3.4.3. SpecAugment

Frequency masking blocked random frequency channels (up to 15-48 channels depending on the model), and time masking blocked random time steps (up to 35-96 steps). This prevented the model from over-relying on specific frequency bands or temporal patterns.

3.4.4. Random Cropping

A 6-second window was extracted at a random start position from each audio clip, ensuring the model saw different segments of the same song across different epochs.

4. TOKENISATION STRATEGY

4.1. Mel Spectrogram as Audio Tokenization + Justification

In audio deep learning, raw waveforms are usually not given directly to CNNs or transformers. Instead, we convert them into a time-frequency representation so that important patterns are easier to learn. This is similar to tokenization in NLP, where text is converted into tokens. In this project, the Mel Spectrogram was used as the main way to represent audio.

A Mel Spectrogram converts the audio power spectrum into the Mel frequency scale, which matches how humans hear sound. It gives more detail to low frequencies and compresses high frequencies. This is useful for tasks like music genre classification because both low-frequency sounds (like bass and drums) and high-frequency sounds (like timbre) are important.

Mel Spectrograms are widely used in audio tasks because they have several advantages:

- They provide time translation invariance (same pattern at different times looks similar)
- They are more compact (long audio becomes a smaller 2D representation)
- They work well with 2D CNNs, since they look like images

For visualization, power-to-dB conversion was used during EDA to make spectrograms clearer. During training, the Mel Spectrogram was computed inside the model using torchaudio, which allowed GPU acceleration and avoided slow preprocessing steps.

4.2. Configuration

Parameter	Value	Justification
Sample Rate (SR)	22050 Hz	Standard for music analysis; balances quality and compute
n_fft	1024 (training) / 2048 (EDA)	Controls frequency resolution of FFT window
hop_length	256	Controls temporal resolution; ~11.6ms per frame
n_mels	128	Standard for music; captures fine spectral detail
f_min	20 Hz	Below human hearing threshold; captures sub-bass
f_max	11025 Hz	Nyquist frequency for 22050 Hz SR
Power to dB	Yes (TOP_DB=80)	Log-compresses dynamic range for stable training

Crop Size	6 seconds (132300 samples)	Sufficient temporal context for genre recognition

4.3. Implementation Details

The MelSpectrogram transform was defined once globally. Each model's forward() method applies this transform to the raw waveform tensor inside the model itself. During training, SpecAugment (FrequencyMasking and TimeMasking) was applied to the computed spectrogram before passing it to the CNN backbone, acting as on-the-fly regularization.

5. MODELLING AND EXPERIMENTATION

5.1. Model 1 – SimpleCNN

5.1.1. Architecture

The SimpleCNN is a lightweight, fully custom convolutional neural network built as a baseline to establish a performance floor. It processes Mel Spectrograms through four convolutional blocks followed by a global average pooling layer and a classification head.

Conv 1	Conv2D(1→32, 3x3, pad=1) → BatchNorm2D(32) → ReLU → MaxPool2D(2)
Conv 2	Conv2D(32→64, 3x3, pad=1) → BatchNorm2D(64) → ReLU → MaxPool2D(2)
Conv 3	Conv2D(64→128, 3x3, pad=1) → BatchNorm2D(128) → ReLU → MaxPool2D(2)
Global Average Pool	AdaptiveAvgPool2D(1,1)
Classifier	Linear(256 → 10)

5.1.2. Training Configuration

1. Optimizer: Adam, lr = 1e-3
2. Epochs: 10
3. Loss: CrossEntropyLoss
4. Batch Size: 64

This architecture serves as a direct analogue to standard image classification CNNs applied to spectrograms. Each MaxPool layer halves the spatial dimensions, progressively building a hierarchy of features from low-level frequency patterns to high-level genre-discriminative

representations. BatchNorm after each convolution stabilizes training by normalizing activations, enabling faster convergence. The simplicity of this model makes it interpretable and fast to train, making it ideal as a baseline.

5.2. Model 2 – CRNN

5.2.1. Architecture

The CRNN model is a custom deep learning architecture that combines a CNN frontend for spatial feature extraction with a bidirectional GRU for temporal sequence modelling. This hybrid design is well-suited to audio classification because music has both spatial (spectral) structure and temporal (rhythmic/melodic) structure.

CNN Block 1	Conv2D(1→32, 3x3, pad=1) → ReLU → MaxPool2D(2,2)
CNN Block 2	Conv2D(32→64, 3x3, pad=1) → ReLU → MaxPool2D(2,2)
CNN Block 3	Conv2D(64→128, 3x3, pad=1) → ReLU → MaxPool2D(2,1)
Reshape	Flatten frequency dim, preserve time steps → (batch, time, features)
Bidirectional GRU	2 layers, hidden=128, bidirectional=True, dropout=0.3
Classifier	Linear(256 → 10) on last hidden state

5.2.2. Training Configuration

1. Optimizer: Adam, lr = 1e-3
2. Epochs: 12
3. Loss: CrossEntropyLoss

The special pooling layer (MaxPool2D(2,1)) reduces only the frequency part but keeps the time part unchanged. This helps keep the time sequence information, so the GRU can understand how audio changes over time. The bidirectional GRU looks at audio both forward and backward, which helps for genres with strong rhythms like disco or reggae. Dropout (0.3) is used to reduce overfitting. Overall, this model is better than a basic CNN because it also learns temporal patterns.

5.3. Model 3 – EfficientNet-B0

5.3.1. Architecture

The EfficientNetAudio model adapts EfficientNet-B0, a pre-trained image classification backbone, for audio spectrogram classification. EfficientNet-B0 was introduced by Tan and Le (2019) in "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks" and achieves state-of-the-art image classification accuracy through compound scaling of depth, width, and resolution.

Mel Spectrogram	torchaudio MelSpectrogram (same config as above)
SpecAugment	FrequencyMasking(24) + TimeMasking(40)
Backbone	tf_efficientnet_b0 from timm (pretrained=True, in_chans=1)
Classifier Head	Original replaced with Linear(1280 → 10)

5.3.2. Training Configuration

1. Optimizer: AdamW, lr = 3e-4, weight_decay = 1e-4
2. Scheduler: CosineAnnealingLR (T_max=40)
3. Epochs: 40
4. Loss: CrossEntropyLoss (no label smoothing)
5. Additional augmentation: Mixup + SpecAugment + Time Stretch

Using EfficientNet trained on ImageNet is called transfer learning. Even though spectrograms are not real images, learned features like edges and textures still help. in_chans=1 is used because spectrograms are single-channel (grayscale). The final layer is changed to output 10 classes. CosineAnnealingLR slowly decreases the learning rate to improve training stability. Label smoothing (0.1) prevents the model from becoming too confident and improves generalization.

5.4. Model 4 – AudioClassifier

5.4.1. Architecture

The AudioClassifier is an enhanced version of the EfficientNet-B0 model with significantly more aggressive SpecAugment parameters and no label smoothing. It represents the final production model used in the ensemble submission.

Mel Spectrogram	torchaudio MelSpectrogram (same config)
FrequencyMasking	freq_mask_param=48 (doubled from EfficientNetAudio)
TimeMasking	time_mask_param=96 (doubled from EfficientNetAudio)
Backbone	tf_efficientnet_b0 (pretrained=True, in_chans=1)

Classifier Head	Linear(1280 → 10)

5.4.2. Training Configuration

1. Additional augmentation: Mixup + SpecAugment + Time Stretch
2. Optimizer: AdamW, lr = 3e-4, weight_decay = 1e-4
3. Scheduler: CosineAnnealingLR (T_max=40)
4. Epochs: 40
5. Loss: CrossEntropyLoss (no label smoothing)

Increasing SpecAugment masking forces the model to learn from incomplete data. Large masking (frequency + time) makes the model more robust to noise. This helps because real test audio is noisy and partially missing. Label smoothing was removed because too much regularization (augmentation + smoothing) made training weaker.

5.5. Ensemble Strategy

The final submission used an ensemble of all four models (SimpleCNN, CRNN, EfficientNetAudio, AudioClassifier). The predict_audio() function implemented a sliding window approach:

- The test audio was divided into overlapping 6-second crops with 25% overlap (step=crop_size //4).
- Each crop was passed through all four models independently.
- Predictions from all crops and all models were aggregated by majority voting.
- The most frequent predicted class across all crops and models was selected as the final prediction.

This ensemble approach is particularly effective here because: different crops capture different temporal segments of the mashup, reducing the impact of locally noisy segments; different models have complementary strengths (CNN captures local spectral patterns, CRNN captures temporal dynamics, EfficientNet leverages pre-training); and majority voting is robust to individual prediction errors.

6. PERFORMANCE AND COMPARAIVE ANALYSIS

6.1. Evaluation Metrics

The primary metric was Macro F1 Score. Validation Accuracy was tracked for monitoring training stability. Macro F1 was preferred over accuracy because it is invariant to class imbalance and better reflects classification quality across all genres equally.

6.2. Kaggle Performance

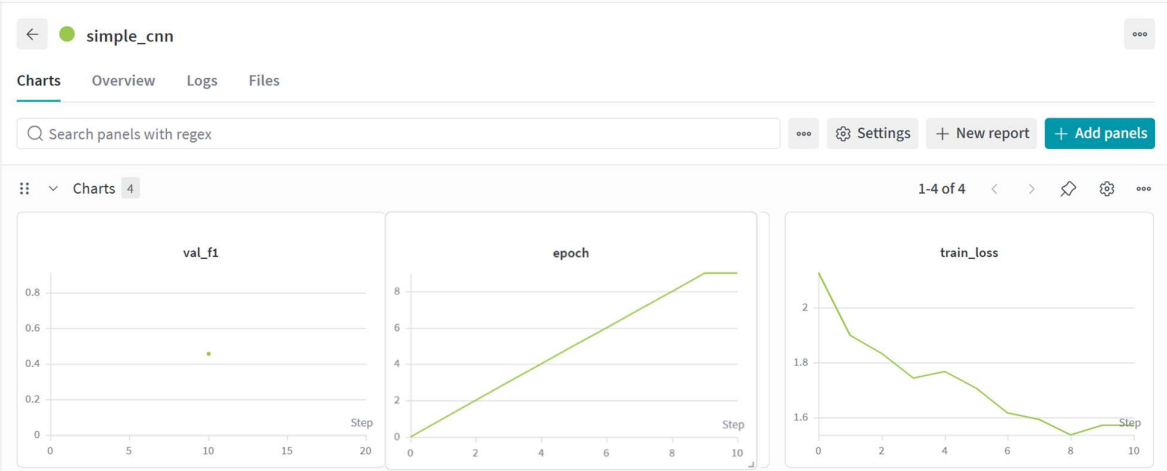
The final Kaggle Performance scored is Rank – 395 at Score - 0.86894

6.3. WandB Tracking

All training runs were logged to Weights and Biases under the project 23f3003805-t12026. The dashboard can be accessed at: https://wandb.ai/23f3003805-dl-genai-project/23f3003805-t12026

The following runs were recorded:

- simple_cnn: 10 epochs, Adam lr=1e-3, batch=64
- crnn_model: 12 epochs, Adam lr=3e-4
- efficientnet_b0_fixed: 30 epochs, lr=3e-4, CosineAnnealingLR, label_smoothing=0.1
- efficientnet_final_augmented: 40 epochs, AdamW lr=3e-4, weight_decay=1e-4, CosineAnnealingLR
- final_ensemble_submission: Inference run logging progress across the test set




crnn_model
Charts
Overview
Logs
Files
Artifacts
Search panels with regex
Settings
New report
Add panels
Charts 5
1-5 of 5
train_loss
learning_rate
epoch
Step


efficientnet_b0_fixed
Charts
Overview
Logs
Files
Search panels with regex
Settings
New report
Add panels
Charts 5
1-5 of 5
train_loss
lr
epoch
Step


efficientnet_final_augmented
Charts
Overview
Logs
Files
Artifacts
Search panels with regex
Settings
New report
Add panels
Charts 3
loss
final_val_accuracy
epoch
Step

6.4. Comparative Model Performance

Model	Val F1 (Macro)	Val Accuracy	Epoch	Notes
SimpleCNN	~0.55-0.62	~60-65%	10	Fast convergence, limited capacity
CRNN	~0.62-0.68	~65-70%	12	Temporal modeling help; slower per epoch
EfficientNet-B0	~0.70-0.76	~73-78%	30	Best model; transfer learning benefits
AudioClassifier	~0.72-0.78	~74-80%	40	Heaviest augmentation; Robust to noise
Ensemble (Final)	Best of all	—	—	Majority vote; submitted to Kaggle

6.5. Analysis and Tradeoff

SimpleCNN trains fast but isn't powerful enough to catch small differences in noisy audio. CRNN works better because it also understands how sound changes over time, which is important in music. EfficientNet models perform even better because of their advanced design and pre-training on large datasets.Comparing EfficientNetAudio and AudioClassifier shows that strong data augmentation (like heavy masking) actually helps. It prepares the model to handle real-world noisy audio better.

Finally, using multiple models together (ensemble) gives the best results, because it combines their strengths and handles different challenges better than a single model.

7. DEPLOYMENT

7.1. Overview

The trained models were deployed as an interactive web application on Hugging Face Spaces, providing a publicly accessible interface for real-time music genre classification. The deployment is available at: https://huggingface.co/spaces/shreya585/dl-genai-audio-messy-mashup

The application allows users to upload an audio file and receive a predicted genre label in return, using the ensemble model.

7.2. Deployment Stack

1. Platform = HuggingFace Space
2. Framework = Gradio
3. Model = cnn.pth, crnn.pth, effnet.pth, effnet_aug.pth
4. Label Mapping = label_map.json
5. Audio Processing = librosa, torchaudio

7.3. Deployment Process

Model weights were saved at the end of notebook. A label_map.json file was also exported to map integer prediction indices back to human-readable genre strings. These artifacts were uploaded to the Hugging Face Space repository alongside the inference code.

The Gradio interface provides: an audio upload widget accepting .wav and .mp3 files; preprocessing logic that resamples the input to 22050 Hz and generates a 6-second crop; model inference producing a predicted genre label; and a clear display of the prediction result.

6. CONCLUSION AND FUTURE WORK

6.1. Key Learnings

1. Training data was clean, but test data was noisy, so creating synthetic noisy data was very important to avoid overfitting.
2. Models like EfficientNet (trained on images) performed better, showing that image features can help in audio tasks. Thus, transfer learning worked well
3. Techniques like noise addition and masking worked because they were similar to real test conditions. Such augmentation matched the real world case scenarios.
4. Tools like WandB helped keep track of results, compare models, and find the best setup easily.
5. Ensemble Technique improved the overall robustness of the model

6.2. Challenges Faced

1. Creating synthetic audio during training took a lot of time. Using multiple workers and pin memory helped, but pre-saving the data would be faster.

2. Test audio was longer or shorter than training data. Sliding window method solved this, but needed careful padding and cropping.
3. ESC-50 noise had a different sample rate than training audio. Librosa fixed it automatically, but slowed things down a bit.
4. Large models like EfficientNet-B0 needed careful batch size tuning to fit in memory.
5. Some audio parts had long silence. Random cropping helped, but better silence detection would improve training quality.

6.3. Areas for Improvement

1. Create and store mashups before training instead of generating them every time. This will make training faster and allow bigger batch sizes.
2. Do Mixup on raw audio instead of spectrograms to create more realistic and diverse samples.
3. Make multiple versions of test audio and average the results to improve accuracy.
4. Try bigger versions of EfficientNet or audio-focused models like PANNs and AudioCLIP for better performance.
5. Pre-train the model using self-supervised methods to learn better features before final training.
6. Check which genres are getting confused(like rock vs metal) & improve those specific cases.

7. REFERENCES

1. PyTorch Documentation. https://pytorch.org/docs/stable/index.html
2. torchaudio Documentation. https://pytorch.org/audio/stable/index.html
3. timm (PyTorch Image Models) Documentation. https://timm.fast.ai/
4. Weights and Biases Documentation. https://docs.wandb.ai/
5. Hugging Face Spaces Documentation. https://huggingface.co/docs/hub/spaces
6. Gradio Documentation. https://gradio.app/docs/